Wykład 5

Biblioteki PyTorcha

Bibilioteki różnych domen PyTorcha mają funkcje do ładownia danych z różnych źródeł

torchvision

This library is part of the PyTorch project. PyTorch is an open source machine learning framework. The torchvision package consists of popular datasets, model architectures, and common image transformations for computer vision.

torchtext

This library is part of the PyTorch project. PyTorch is an open source machine learning framework. The torchtext package consists of data processing utilities and popular datasets for natural language.

torchaudio

Torchaudio is a library for audio and signal processing with PyTorch. It provides I/O, signal and data processing functions, datasets, model implementations and application components.

TorchRec

TorchRec is a PyTorch domain library built to provide common sparsity & parallelism primitives needed for large-scale recommender systems (RecSys). It allows authors to train models with large embedding tables sharded across many GPUs.

torchtune

A PyTorch-native library designed for fine-tuning large language models (LLMs). torchtune provides supports the full fine-tuning workflow and offers compatibility with popular production inference systems.

Pre-built Datasets and Functions

```
torchvision.datasets
torchaudio.datasets
torchtext.datasets
torchrec.datasets
```

kaggle

Search

Create

Home

Competitions

Datasets

Models

Code

Discussions

Learn

More

Datasets

Search 4,060 datasets

Computer Vision ×

4,060 Datasets

Bird vs Drone

Locked_in_hell · Updated a month ago

Usability **9.4** · 41907 Files (other) · 1 GB

About ImageNet

News and updates

- March 11 2021: ImageNet website update and a new paper on privacy preservation.
- October 10, 2019: The ILSVRC 2012 classification and localization test set has been updated. The Kaggle challenge and our download page both now contain the updated data.
- September 21, 2019: ImageNet 10th Birthday Party
- September 17, 2019: Research update on filtering and balancing the ImageNet person subtree
- June 2019: ImageNet wins the PAMI Longuet-Higgins Prize (Retrospective Most Impactful Paper from CVPR 2009)
- July 2017: ImageNet: Where have we been? Where are we going? at a CVPR 2017 workshop.
- July 2017: ImageNet is covered by Quartz.
- June 2, 2015: Follow-up update regarding status of the server
- May 19, 2015: Annoucement regarding the submission server

Overview

Welcome to the ImageNet project! ImageNet is an ongoing research effort to provide researchers around the world with image data for training large-scale object recognition models.

What is ImageNet?

ImageNet is an image dataset organized according to the WordNet hierarchy. Each meaningful concept in WordNet, possibly described by multiple words or word phrases, is called a "synonym set" or "synset". There are more than 100,000 synsets in WordNet; the majority of them are nouns (80,000+). In ImageNet, we aim to provide on average 1000

Image classification

`Caltech101`(root[, target_type, transform, ...])	Caltech 101 Dataset.
`Caltech256`(root[, transform, ...])	Caltech 256 Dataset.
`CelebA`(root[, split, target_type, ...])	Large-scale CelebFaces Attributes (CelebA) Dataset Dataset.
`CIFAR10`(root[, train, transform, ...])	CIFAR10 Dataset.
`CIFAR100`(root[, train, transform, ...])	CIFAR100 Dataset.
`Country211`(root, ~pathlib.Path], split, ...)	The Country211 Data Set from OpenAI.

CIFAR-10 i CIFAR-100 zostały utworzone przez Alexa Krizhevsky, Vinod Nair i Geoffrey Hinton.

10 klas obiektów

Zbiór danych jest podzielony na pięć paczek treningowych i jedną partię testową, z których każda zawiera 10000 obrazów. Paczka testowa zawiera dokładnie 1000 losowo wybranych obrazów z każdej klasy. Paczki treningowe zawierają pozostałe obrazy w losowej kolejności, ale niektóre paczki treningowe mogą zawierać więcej obrazów z jednej klasy niż z innej. W sumie paczki treningowe zawierają dokładnie 5000 obrazów z każdej klasy.

DataLoader

PyTorch daje możliwość skorzystania z dwóch klas do obróbki danych: ***torch.utils.data.Dataset*** oraz ***torch.utils.data.DataLoader***.

Zadaniem *Dataset* jest pobranie ze zbioru pojedynczej danej wraz z jej opisem (label), natomiast *DataLoader* otacza dane pobrane przez *Dataset* iteratorem, zapewnia serwowanie ich w batchach, działanie w wielu wątkach, aby przyspieszyć pobieranie danych do uczenia, jak również takie operacje jak choćby mieszanie danych.

https://manduk.ai/pl/przygotowanie-danych-do-uczenia-maszynowego-w-pytorch/

https://manduk.ai/pl/pytorch-podzial-zbioru-transformacje-uczenie-na-gpu-oraz-wizualizacja-metryki/

DataLoader

```
from torch.utils.data import DataLoader

train_dataloader = DataLoader(training_data, batch_size=64, shuffle=True)
test_dataloader = DataLoader(test_data, batch_size=64, shuffle=True)
```

Powinno być False

The CIFAR-10 dataset

kodu napisanego przez Gemini Prześlij

- bin
- boot
- content
- data
- cifar-10-batches-py
  - batches.meta
  - data_batch_1
  - data_batch_2
  - data_batch_3
  - data_batch_4
  - data_batch_5
  - readme.html
  - test_batch

25 s

```
from torchvision import datasets
from torchvision import transforms
from torch.utils.data import DataLoader

# Pobranie zbioru CIFAR10
dataset_train = datasets.CIFAR10('/data', train=True, download=True,
                                 transform=transforms.Compose([transforms.ToTensor()]))

dataset_test = datasets.CIFAR10('/data', train=False, download=True,
                                transform=transforms.Compose([transforms.ToTensor()]))

# Create test and traing dataloader with dataset
train_loader = DataLoader(dataset_train, batch_size=64, shuffle=True)

test_loader = DataLoader(dataset_test, batch_size=64, shuffle=False)
```

```
100%|██████████| 170M/170M [00:02<00:00, 64.3MB/s]
```

Patrz Obróbka zdjęćPIL

```
print(f"Dataloaders: {train_loader, test_loader}")
print(f"Długość uczącego dataloader: {len(train_loader)} paczek")
print(f"Długość testowego dataloader: {len(test_loader)} paczek/batches")
```

```
Dataloaders: (<torch.utils.data.dataloader.DataLoader object at 0x7d1d1bfd5710>, <tor
Długość uczącego dataloader: 782 paczek
Długość testowego dataloader: 157 paczek/batches
```

157*64=10048
782*64=50048

Zmiana wymiaru paczki

```
# Create test and traing dataloader with dataset
train_loader = DataLoader(dataset_train, batch_size=32, shuff

test_loader = DataLoader(dataset_test, batch_size=32, shuffle=
```

```
print(f"Dataloaders: {train_loader, test_loader}")
print(f"Długość uczącego dataloader: {len(train_loader)} pacze
print(f"Długość testowego dataloader: {len(test_loader)} pacze
```

```
Dataloaders: (<torch.utils.data.dataloader.DataLoader object a
Długość uczącego dataloader: 1563 paczek
Długość testowego dataloader: 313 paczek/batches
```

```
!pip install efficientnet_pytorch
from torch import nn
from torch import optim
from efficientnet_pytorch import EfficientNet
model = EfficientNet.from_pretrained('efficientnet-b0', num_classes=10)
```

```
# Definicja straty i optymalizacji
criterion = nn.CrossEntropyLoss()
optimizer = optim.RMSprop(model.parameters(), lr=0.0002)

# Pętla ucząca z Loaderem
for x, y in train_loader:
  optimizer.zero_grad()
  pred = model(x)
  loss = criterion(pred,y)
  loss.backward()
  optimizer.step()

  print (loss.item())
```

```
# Testowanie test_loader
totalLoss=0
for x, y in test_loader:
  pred = model(x)
  loss = criterion(pred,y)
  totalLoss+=loss.item()
```

ImageFolder

PyTorch ma wiele wbudowanych funkcji ładowania danych dla popularnych typów danych.

ImageFolder jest pomocny, jeśli nasze obrazy są w standardowym formacie klasyfikacji obrazów.

CLASS torchvision.datasets.ImageFolder(*root: ~typing.Union[str, ~pathlib.Path], transform: ~typing.Optional[~typing.Callable] = None, target_transform: ~typing.Optional[~typing.Callable] = None, loader: ~typing.Callable[[str], ~typing.Any] = <function default_loader>, is_valid_file: ~typing.Optional[~typing.Callable[[str], bool]] = None, allow_empty: bool = False*) [SOURCE]

A generic data loader where the images are arranged in this way by default:

```
root/dog/xxx.png
root/dog/xxy.png
root/dog/[...]/xxz.png

root/cat/123.png
root/cat/nsdf3.png
root/cat/[...]/asd932_.png
```

Nazwa folderu jest etykietą klasy

```
pizza_steak_sushi/ <- overall dataset folder
    train/ <- training images
        pizza/ <- class name as folder name
            image01.jpeg
            image02.jpeg
            ...
        steak/
            image24.jpeg
            image25.jpeg
            ...
        sushi/
            image37.jpeg
            ...
    test/ <- testing images
        pizza/
            image101.jpeg
            image102.jpeg
            ...
        steak/
            image154.jpeg
            image155.jpeg
            ...
        sushi/
            image167.jpeg
            ...
```

Korzystanie z wytrenowanych modeli

Models and pre-trained weights

The `torchvision.models` subpackage contains definitions of models for addressing different tasks, including: image classification, pixelwise semantic segmentation, object detection, instance segmentation, person keypoint detection, video classification, and optical flow.

```
from torchvision.models import resnet50, ResNet50_Weights

# Old weights with accuracy 76.130%
resnet50(weights=ResNet50_Weights.IMAGENET1K_V1)

# New weights with accuracy 80.858%
resnet50(weights=ResNet50_Weights.IMAGENET1K_V2)

# Best available weights (currently alias for IMAGENET1K_V2)
# Note that these weights may change across versions
resnet50(weights=ResNet50_Weights.DEFAULT)

# Strings are also supported
resnet50(weights="IMAGENET1K_V2")

# No weights - random initialization
resnet50(weights=None)
```